Object-oriented programming

Tutorial 1. Classes and object (Part 1)

Exercise 1: First approach to BlueJ and introduction to class and object concepts.

("Shapes" and "Picture" projects, from Mickael Kolling's "Objects First With Java")

Objects, state, methods.

1. Launch BlueJ and open the Shapes project (go to Project →Open project and select the directory where BlueJ is installed on your machine) (ex. C:\Programmes\BlueJ), A diagram appears in the window. Each colored rectangle in the graph represents a class in our project. We have the following classes: Circle, Square, Triangle and Canvas.
2. Right-click on Circle and select new Circle(). A window appears, asking you for a name for the circle you've created. You can keep the default name, or change it if you wish. A red rectangle appears at the bottom of the screen. It represents the object you created.

   By convention, class names begin with an uppercase letter and object names begin with a lowercase letter.
3. Create another Circle object and another Square object.
4. Right-click on one of the Circle objects, and you'll see a context menu with several method names. Choose *makeVisible* method. This will draw the circle in a separate window. Several other operations are possible on circles. Try calling the *moveRight* and *moveDown* methods. You can also hide or show the circle.
5. Call the *moveHorizontal* method. A dialog box appears, asking for a value. Type 50 and click OK. The circle will move 50 pixels to the right. The *moveHorizontal* method is written in such a way as to require information before it can be executed. This information is the distance by which the circle is to be moved. This method is more flexible than *moveRight* and *moveDown*, which move the circle a fixed distance.
6. Call the *moveVertical*, *showMovevertical* and *changeSize* methods.
7. Invoke the *changeColor* method on one of the Circle objects by entering the string "red", then try other colors. What happens if you enter a string without quotation marks?
8. In BlueJ, you can inspect the state of an object by selecting the Inspect function from the context menu. A window appears showing the object's current state
9. Inspect the states of the various objects you've created. Try changing the state of an object (by calling a *moveLeft* or *moveRight* method, for example) and check whether the state has changed.
10. Use Circle Square and Rectangle objects to create a figure representing a house and a sun.

Object-oriented programming

Interaction between objects, source code

11. Close the Shapes project and open the Picture project
12. Create an instance of the Picture class and invoke its draw method. Invoke the *setBlackAndWhite* and *setColor* methods.
13. In the context menu of the *Picture* class, you will find the Open editor option. Select this option, and an editor will open and display the class source code (this can be done by double-clicking on the class).

    The source code, written in Java, defines the class's attributes and methods and determines what happens when a method is invoked. If the source code is modified, the class appears hatched and needs to be recompiled.

14. In the Picture class source code, locate the section responsible for drawing the image and modify it to make the sun blue instead of yellow.
15. Add a second sun to the image.
16. Create a sunset effect for an image with a single sun, making it gradually descend towards the bottom.

Exercice 2

1. First class: the Point class

1. Open your IDE and create a project called PointProject containing the Point class seen in class.

```
class Point {
private int x;
private int y;
public Point(int absc, int ord){
x = absc;
y = ord;
}
public void move (int dx, int dy){
x = x + dx;
y = y + dy;}
public void display(){
System.out.println("My coordinates are x = "+x+" and y ="+y);
}
}// End of the Point class
```

If you try to run the Point file as it is, the compiler will report an error. In fact, just like in the C language, running a Java program consists of executing the body of a main method, which must be inside a class. A Java file can contain multiple classes, but only one can include the main method.

In our case, the Point.java file contains only the Point class.

2. Add a TestPoint class containing only the following main method

```
public static void main (String args [] ){
Point p = new Point(2,3);
p.display();
}
```

A few explanations:

a. The `static` keyword indicates that the `main` method is a class method, meaning it is not bound to any particular instance (object) of the `Point` class.

b. The `String args[]` parameter of the `main` method is an array of `String` objects (character strings) used to retrieve arguments passed to the program when it is launched. It is possible to run a program without providing arguments (as is the case in our `Point` program), but the `String args[]` declaration is mandatory. You can write `String args[]` or `String[] args`. You can also choose another name for the `args` array, such as `data` or `param`, but the convention is to use `args`.

c. The `public` keyword in `public static void main` is required for the program to execute. It allows the virtual machine to access the `main` method. Note that the order of the `public` and `static` keywords can be reversed.

B. Execution with Command-Line Parameter Passing

The `Point` program, as it is, has no real functionality. We want to enhance it by allowing the creation of `Point` objects, where the `x` and `y` values (abscissa and ordinate) will be read from user input. We will see how to assign values to these attributes using command-line arguments.

Back the `main` method: as explained above, the `args` array contains objects of type `String`, which allows us to retrieve arguments passed to the `main` method through the command line. We can access these parameters by writing `args[0]`, `args[1]`, etc. The number of parameters is provided by `args.length`. We can now pass the values for `x` and `y` as arguments. However, be aware that these arguments are of type `String`, which means they are character strings. Since our `x` and `y` attributes are integers, we need to convert these `String` arguments into `int` variables. This can be done using the `parseInt` method from the `Integer` class.

1. Modify the *main* method of the class *TestPoint* as follow :

```
class TestPoint{
        public static void main (String args []){
                int x = Integer.parseInt(args[0]);
                int y = Integer.parseInt(args[1]);
                Point p = new Point (x,y);
                p.display();
                }
    }
```

2. Recompile and run the program, by passing command-line values.

To read data from the command line, you have several options depending on the execution mode:

- If you are using the console, the command *javac TestPoint.java* is used to compile the source code, and *java TestPoint 3 5* is used to run it with the arguments 3 and 5.
- If you are using BlueJ, right-click on the *TestPoint class*, select the main method to execute it, and enter the coordinates in quotes (so they are considered as strings) in the appropriate fields.
- If you are using Eclipse, go to the Run > Run Configuration menu. A window will appear. Choose the "Arguments" tab. Enter your arguments in the "Program Arguments" text box, separated by a space, e.g., 3 5.
- In another IDE, you should look for the menu that allows you to enter parameters, as in Eclipse.

C. Reading Data from the Keyboard

We now want to further modify our program by allowing the user to input data from the keyboard. To do this, Java provides several methods. The simplest one was introduced in version 1.5, and it involves using an object from the `Scanner` class. To do this, we must first import the `Scanner` class, which is located in `java.util`, using the `import` statement, which should be placed at the beginning of your program:

```
import java.util.Scanner
class TestPoint{… }
```

In the main method, add this line to declare a Scanner object:

```
public static void main (String args []){
        Scanner sc = new Scanner(System.in);
       // … }
```

This object can take different parameters. We use the one corresponding to the standard input (keyboard) in Java, System.in. (When you write System.out.println(), you are calling the println() method on the standard output.)

To allow Java to capture what you will input, you can use the following methods:

- next(): returns the next string. By default, string delimiters are space characters (space, newline, tab, etc.).
- nextLine(): returns the next line or the remainder of the current line.
- nextInt(): for this method to work, the next input must be an integer, which the method will return; otherwise, the method will throw an exception.
- nextLong(), nextByte(), nextDouble(), etc.: These work the same way as nextInt, but the type of the

input data must match the type expected by the method.

In our example, since the attributes are integer, we will use nextInt:

```
public static void main (String args []){
    Scanner sc = new Scanner(System.in);
    System.out.println("Please enter the value of x :");
    int x = sc.nextInt();
    System.out.println("Please enter the value of y :");
    int y = sc.nextInt();
    Point p = new Point(x,y);
    p.display();
}
```

Exercice 3

1. Create a class *Student* with the attributes *name, surname,* and *idNumber*. The *idNumber* is a unique integer assigned during student registration and automatically incremented each time a new student is created.
2. Provide a full constructor to initialize all attributes, as well as a *void display()* method that prints the values of the student's attributes.
3. Add three new attributes *mod1, mod2*, and *mod3* to represent the student's averages in three modules. Implement a *double calculateAverage()* method that returns the student's overall average, assuming all modules have the same coefficient.
4. Modify your program to allow the registration of a student by directly providing the averages of the three modules (case of a student who has transferred from another institution).
5. Create a class *Test* with no attributes, containing only the main method.
6. Test your program by creating both new students and students transferred from another institution.
7. Create a class named *Administration* responsible for managing a collection of students, with a maximum capacity of 500 students. The program should allow:
    - adding new students or transferred students and storing them in a static array,
    - displaying the list of all students,
    - displaying the list of students whose average is above a threshold entered by the user.
8. Test your program by creating an object of the *Administration* class and several instances of the *Student* class, and by invoking the different methods.

Object-oriented programming

9. Modify the *Student* class so that the module averages are stored in a three-element array. What is the impact of this modification on your program? Did you need to modify the Administration class? Why?